Universidad Nacional Autónoma de México

Facultad de Ingeniería

Integrantes:

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Sistemas Distribuidos

Grupo: 01 - Semestre: 2026-2

Práctica:

RPC y gRPC

Profesora:

M.I. Elba Karen Saenz García

Fecha de Entrega:

4 de Mayo del 2026

Índice

1 Objetivos **1**

2 Investigación Previa **1**

2.1 Que és IDL, XDR, y Protocol Buffers 1

2.2 Tipos de datos a usar en XDR y Protocol Buffers 5

2.3 Significado de los números en archivos .x y .proto 6

3 Desarrollo **7**

4 Actividad 1 . . . . . **7**

4.1 Diseño y Justificación de la Interfaz (IDL) 7

4.2 Generación, Compilación y Ejecución del Proyecto 8

4.3 Explicacion del codigo del servidor 9

4.4 Desarrollo del Cliente 14

4.5 Resultados: 15

4.5.1 Opcion 1 (Registro de alumnos): 15

4.5.2 Opcion 2 (Buscar alumno): 16

4.5.3 Opcion 3 (Actulizacion de alumno): 16

4.6 Opcion 4 (Eliminar alumno:) 16

5 Actividad 2 . . . . . **16**

5.1 Planteamiento de la Solución 16

5.2 Diseño y Justificación del Archivo `.proto` 17

5.2.1 La Estructura Principal: `Alumno` (Mensajes) 19

5.2.2 Las Funciones del Servidor (Servicio y Métodos RPC) 20

5.3 Desarrollo del Lado del Servidor 22

5.3.1 1. Concurrencia y Persistencia Segura (I/O) 22

5.3.2 2. Lógica de Negocio (Clase `AlumnoServidor`) 23

5.3.3 3. Inicialización Multi-hilo 24

5.4 Desarrollo del Lado del Cliente 25

5.4.1 1. Conexión Dinámica y Canal gRPC 25

5.4.2 2. Empaquetado y Robustez (Manejo de Errores) 26

5.4.3 3. Consumo de Streaming de Datos 27

5.5 Resultados de la Actividad 2 28

5.5.1 Local 28

5.5.2 Remoto 33

Referencias **37**

1. Objetivos

- Comprender el uso del IDL de Sun-RPC (XDR), el funcionamiento del compilador de interfaces (rpcgen) y el desarrollo de clientes y servidores que hagan uso de las llamadas remotas.
- Comprender el uso de grpc con Python comprendan cómo y definir interfaces de servicios con Protocol Buffers (.proto)

2. Investigación Previa

2.1. Que és IDL, XDR, y Protocol Buffers

IDL (Lenguaje de Definición de Interfaces)

Un Lenguaje de Definición de Interfaces (IDL) es más que un simple formato de texto, es como un contrato entre diferentes sistemas informáticos. En los sistemas distribuidos donde un programa en Python en una computadora necesita pedirle datos a un servidor en C++ en otra computadora, ambos sistemas hablan idiomas diferentes en la memoria. El IDL sirve como un puente universal que describe qué funciones están disponibles para llamarse a través de la red y qué forma exacta tienen los datos que van a viajar.

El verdadero de un IDL esta en su flujo de trabajo automatizado, el cual consta de dos fases principales:

1. **Definición:** El programador escribe un archivo abstracto (por ejemplo, myspec.idl) donde define únicamente las firmas de las funciones (nombre, parámetros y tipo de retorno), sin escribir nada de lógica interna.
2. **Compilación:** Este archivo se pasa por un Çompilador de IDL". El compilador lee el archivo neutro y genera automáticamente código fuente en los lenguajes de programación de destino.

Este código generado se divide en dos piezas clave para la arquitectura de Llamada a Procedimiento Remoto (RPC):

- **El Client Stub (Representante del Cliente):** Es un código que se integra en la aplicación del usuario. Actúa como un proxy local; cuando el cliente llama a una función, el stub intercepta la llamada, empaqueta los datos (proceso de marshaling) y los envía por la red.
- **El Server Skeleton (Esqueleto del Servidor):** Es el código que se integra en el servidor. Su trabajo es escuchar la red, recibir el paquete, desempaquetarlo (proceso de unmarshaling) y pasárselo a la función real que el desarrollador escribió en el servidor.

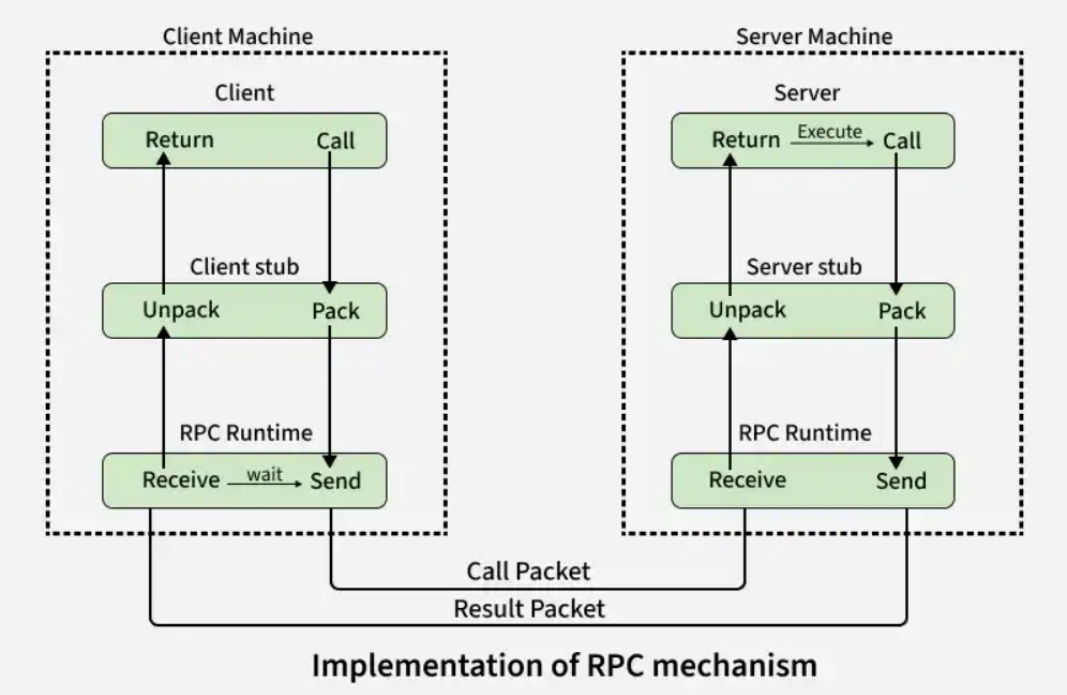

Figura 1: Flujo completo del mecanismo de Llamada a Procedimiento Remoto (RPC), mostrando el empaquetado (marshalling) en el cliente, la transmisión a través del RPC runtime y la ejecución en el servidor mediante el server skeleton.

XDR (Representación Externa de Datos)

La Representación Externa de Datos (XDR) es un estándar de serialización de datos puro y duro. Fue concebido en la década de 1980 por Sun Microsystems y fue la tecnología que hizo posible el famoso sistema de archivos en red NFS (Network File System). Hoy en día está estandarizado bajo el RFC 4506 de la IETF.

El propósito de XDR es resolver el problema de la variedad del hardware, diferentes procesadores almacenan los números en distinto orden (conocido como endianness, como little-endian de Intel vs big-endian de Motorola) y alinean la memoria de formas distintas. Si enviáramos la memoria cruda de una computadora a otra, la segunda interpretaría basura, para solucionarlo, XDR pone unas reglas rígidas sobre cómo convertir cualquier tipo de dato a un formato universal de bytes que pueda ser entendido por cualquier máquina, sin importar su arquitectura:

- **Tipado Implícito:** A diferencia de JSON o XML que son formatos de “tipado explícito"que envían el nombre de la variable junto con el dato, ej: “edad": 25, XDR es de “tipado implícito". En XDR, no viajan metadatos ni etiquetas por el cable, si no que viajan puros ceros y unos. Esto ahorra ancho de banda y tiempo de procesamiento, pero obliga a que tanto el cliente como el servidor tengan exactamente el mismo mapa mental (el archivo .x) para saber cómo leer el flujo de bytes ciego que llega.

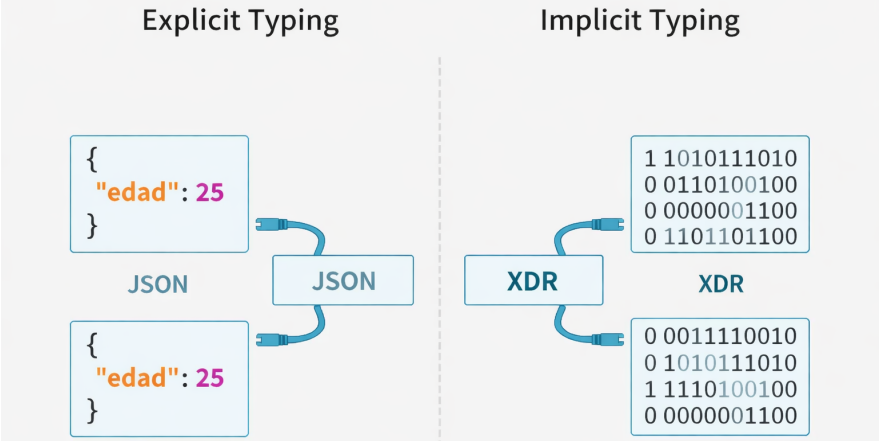

Figura 2: Comparación entre tipado explícito e implícito en serialización de datos.

- **La Regla de Oro de los 4 Bytes:** XDR exige que la unidad básica de todo dato transmitido sea un bloque múltiplo de 4 bytes (32 bits). Los ingenieros eligieron 4 bytes debido a que 2 bytes causaban problemas de alineación en computadoras de la época, y 8 bytes desperdiciaban demasiado espacio.
- **Relleno (Padding):** Si se quiere enviar un tipo de dato pequeño como un simple carácter de 1 byte, XDR no lo envía solo. Forzosamente le inyecta 3 bytes adicionales llenos de ceros (padding) al final para completar el bloque de 4 bytes. Esto garantiza que los datos se alineen perfectamente en la memoria de casi cualquier procesador del mundo, permitiendo que las lecturas sean extremadamente rápidas, aunque sacrifique un poco de espacio en la red.

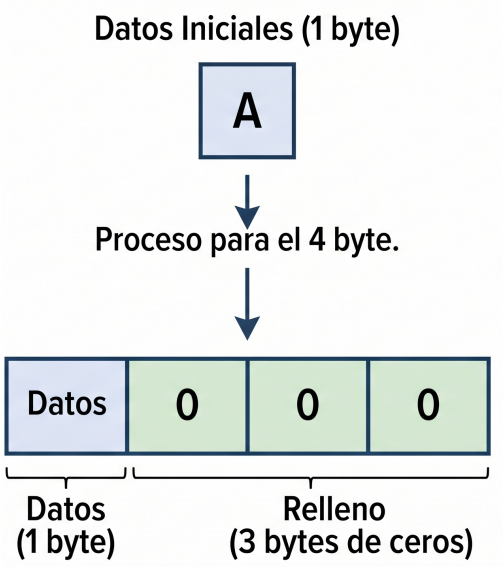

Figura 3: Padding o relleno

Protocol Buffers (Protobuf)

Protocol Buffers es el estándar moderno de facto para microservicios. Fue creado por Google para uso interno y luego liberado al público. Protobuf funciona como un IDL (a través de sus archivos .proto) y también es el propio algoritmo de serialización binaria. Es el motor principal detrás de gRPC, el framework de comunicación de alto rendimiento de Google.

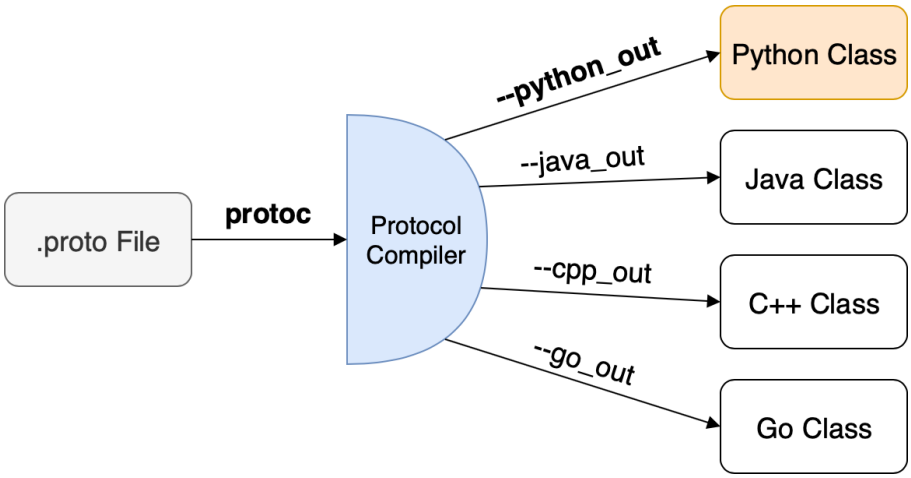

Figura 4: Proceso de compilación en Protocol Buffers, un archivo `.proto` es procesado por el compilador `protoc` para generar clases en distintos lenguajes de programación como Python, Java, C++ y Go.

Mientras XDR se basa en bloques rígidos de tamaño fijo y posiciones exactas, Protobuf fue diseñado para ser compacto, asimétrico y elástico. Google necesitaba enviar miles de millones de mensajes entre sus servidores usando el menor ancho de banda posible, y al mismo tiempo, poder actualizar los programas añadiendo o quitando campos en los mensajes sin que los servidores viejos colapsaran al no entender los nuevos campos y lo logró mediante dos innovaciones:

1. **Esquema Tag-Length-Value (TLV):** En lugar de depender de que las variables lleguen en un orden estricto (como XDR), Protobuf anticipa cada variable con una pequeña “Etiqueta” (Tag). Esta etiqueta le dice al receptor: “El siguiente dato es la variable número 1, y es de tipo entero". Si un servidor viejo recibe un mensaje con la etiqueta “variable número 5"que aún no conoce, simplemente lee la longitud del dato, lo salta sin fallar, y sigue leyendo el resto.

2. **Compresión de Enteros Dinámicos (Varints):** En lugar de desperdiciar 4 bytes estáticos para enviar el número 1 como hace XDR obligatoriamente, Protobuf utiliza enteros de longitud variable (Varints). El sistema manipula los bits de tal forma que los números pequeños ocupan solo 1 byte en la red, mientras que números inmensos pueden llegar a ocupar hasta 10 bytes. Para lograrlo, el bit más significativo de cada byte se usa como un semáforo. Si es 1, significa “el número continúa en el siguiente bytez si es 0, significa “el número termina aquí".

2.2. Tipos de datos a usar en XDR y Protocol Buffers

En XDR (RFC 4506)

El lenguaje de XDR se asemeja bastante a C y define los siguientes tipos para su transmisión:

- **Numéricos:** Enteros de 32 bits con y sin signo (`int`, `unsigned int`), enteros largos de 64 bits (`hyper`, `unsigned hyper`), y valores de punto flotante de precisión simple y doble (`float`, `double`).
- **Lógicos:** El tipo booleano (`bool`), el cual también se transmite ocupando un bloque de 32 bits.
- **Estructurados y Compuestos:**
  - Cadenas de caracteres (`string`).
  - Datos binarios sin procesar (`opaque`), ya sea de longitud fija o variable.
  - Colecciones (`arrays` de longitud fija o variable) y estructuras (`struct`).
  - Uniones discriminadas (`union`): A diferencia de las uniones en C, estas asocian directamente un indicador inicial (un número entero o enumeración) llamado "discriminante" que indica qué tipo de estructura se transmitirá a continuación dentro de la unión.
  - Vacío (`void`): Una cantidad de cero bytes, útil para cuando no hay entrada o salida.

En Protocol Buffers (proto3)

El catálogo de tipos está diseñado para maximizar la eficiencia binaria:

- **Estándares de longitud variable:** `int32`, `int64`, `uint32`, `uint64`. El tamaño en bytes que ocupan en red depende de la magnitud del número (varints).
- **Optimizados para signos negativos:** `sint32` y `sint64`. Emplean la codificación "ZigZag" que hace que los números negativos ocupen muchísimo menos espacio que si se enviaran usando el `int32` regular.
- **De tamaño fijo:** `fixed32`, `fixed64`, `sfixed32`, `sfixed64`. Siempre ocupan 4 u 8 bytes exactos. Resultan más eficientes que los varints solo si se transmiten rutinariamente números increíblemente grandes (superiores a 2^{28}).
- **Otros:** Valores de punto flotante (`float`, `double`), booleanos (`bool`), cadenas de texto (obligatoriamente `string` codificado en UTF-8 o ASCII), y arreglos binarios arbitrarios (`bytes`).

2.3. Significado de los números en archivos .x y .proto

A pesar de que ambos utilizan números, el propósito arquitectónico de dichos dígitos es muy diferente en cada ecosistema.

Números en el archivo .x (XDR / ONC RPC)

En el lenguaje RPCL utilizado en XDR, las solicitudes de red hacia procedimientos remotos se enrutan e identifican inequívocamente gracias a una tupla de tres números: (número de programa, número de versión, número de procedimiento).

1. **Número de Programa:** Identifica una agrupación de funciones remotas afines (la aplicación en general). Existen rangos de asignación oficiales regulados por la IANA para evitar colisiones en la red (por ejemplo, el rango hexadecimal `20000000` a `3fffffff` está destinado a ser definido libremente por el usuario).

2. **Número de Versión:** Permite que un mismo programa tenga múltiples versiones conviviendo en el mismo servidor de forma simultánea. Esto ayuda a no romper la compatibilidad con clientes antiguos cuando los desarrolladores añaden nuevos procedimientos.

3. **Número de Procedimiento:** Señala puntualmente la función o método individual que se va a ejecutar en el servidor remoto. Como convención del estándar, el procedimiento `0` suele ser un procedimiento nulo (NULL) que no recibe ni envía datos, usado simplemente para hacer "ping" y probar si el servidor está en línea.

Números en el archivo .proto (Protocol Buffers)

En Protobuf, a cada variable declarada dentro de un mensaje se le debe asignar estrictamente un Número de Campo.

1. **Reemplazo del nombre del campo:** A diferencia de JSON o XML, que repiten el nombre de la variable en el archivo ("nombre", "edad", etc.), Protobuf nunca envía el nombre de la variable en la red. En su lugar, envía únicamente el número de campo para que el receptor sepa en qué ranura encajar el dato, ahorrando enormes cantidades de ancho de banda.

2. **Economía de espacio:** Los números de campo del 1 al 15 ocupan solamente un byte de almacenamiento al serializarse. Por regla de optimización, estos números de oro deben reservarse exclusivamente para las variables que más se usan en una aplicación. Los números del 16 al 2047 pasan a costar dos bytes.

3. **Reglas estrictas de inmutabilidad y exclusión del 0:** Los números de campo deben ser enteros positivos. El número `0` está prohibido. Además, una vez que un número ha sido asignado a un campo y el sistema está en producción, ese número no se puede cambiar ni se debe reutilizar en caso de que el

campo sea borrado. Reutilizar un número generaría ambigüedad en el descifrado, corrompiendo datos, provocando fallos en el análisis, y exponiendo potencialmente información confidencial entre sistemas que usan distintas versiones del archivo.

3. Desarrollo

4. Actividad 1

4.1. Diseño y Justificación de la Interfaz (IDL)

El archivo inicial es un .x, donde es llamado `alumnos.x`, este archivo establece la estructura de datos y los procedimientos remotos del servidor.

```
struct alumno {
    int id;
    string nombre<>;
    string apellido<>;
    int edad;
    string curso<>;
};

struct busqueda {
    int id;
    string nombre<>;
    string curso<>;
};

union resultado_busqueda switch (int status) {
    case 0:
        alumno al;
    default:
        void;
};

struct alumno_nodo {
    alumno al;
    alumno_nodo *sig;
};

typedef alumno_nodo *lista_alumnos;

```

```
    program REGISTROALUMNOS {
        version REGISTROALUMNOS_V1 {
            bool registrar_alumno(alumno) = 1;
            resultado_busqueda buscar_alumno(busqueda) = 2;
            bool actualizar_alumno(alumno) = 3;
            bool eliminar_alumno(int) = 4;
            lista_alumnos listar_alumnos(void) = 5;
        } = 1;
    } = 0x20000001;
```

En la estructura `alumno`, los campos son el nombre, apellido y curso se definieron como `string`<>. Esta parte es por que indica que son cadenas de longitud variable, lo cual como se vio en la investigacion previa, este metodo y forma de usar lo strings de forma variable optimiza la transmisión de datos porque XDR solo empaqueta los bytes reales de la cadena más su longitud, en lugar de reservar arreglos estáticos grandes que desperdiciarían memoria.

La estructura `busqueda` se baso solo en los campos que se asignaron en la actividad, agrupando únicamente el id, el nombre y el curso.

Para la parte del manejo de respuestas, se implementó la estructura condicional `union resultado_busqueda`. Esta parte utiliza una variable de estado (status) como discriminante. Cuando la búsqueda es exitosa, y por consiguiente el estado es 0, la unión pasa los datos completos del alumno encontrado. Para cualquier otro estado, entonces se pasa un vacio, un (`void`).

Debido a que Sun-RPC no soporta originalmente la parte de listas dinámicas en C como retornos directos, se definió un nodo de lista enlazada llamado `alumno_nodo`. Este nodo contiene la estructura del alumno y un puntero `sig`, que es el que apunta al siguiente nodo del sigueinte alumno.

Por ultimo, el bloque `program` define la agrupación de todas estas llamadas. Se le asignó el número hexadecimal `0x20000001`, esto en referencia a la estructura propuesta originalmente en el documento asignado.

Seccion para el readme.md:

4.2. Generación, Compilación y Ejecución del Proyecto

El compilador de protocolos nativo procesa el archivo de interfaz para generar los esqueletos en lenguaje C.

```
    rpcgen -a -C alumnos.x
```

La bandera `-a` le indica al compilador que debe generar todos los archivos auxiliares posibles, incluyendo plantillas para el cliente, servidor y un archivo Makefile. La bandera `-C` asegura que el código generado cumpla con el estándar ANSI C moderno.

Debido a que en las distribuciones actuales de Linux las rutinas de RPC fueron extraídas de la biblioteca estándar, es obligatorio utilizar la biblioteca `libtirpc` (Transport Independent RPC). Por esto, el proceso de compilación requiere vincular explícitamente estas dependencias.

```
gcc -c alumnos_client.c -I/usr/include/tirpc
gcc -c alumnos_server.c -I/usr/include/tirpc
gcc alumnos_client.o alumnos_clnt.o alumnos_xdr.o -o cliente -ltirpc
gcc alumnos_server.o alumnos_svc.o alumnos_xdr.o -o servidor -ltirpc
```

Durante la compilación de los códigos fuente, se utiliza la directiva `-I/usr/include/tirpc` para que el compilador encuentre los archivos de cabecera correctos. Durante el enlazado final, la instrucción `-ltirpc` incluye los binarios de la biblioteca para resolver las funciones de red. Para ejecutar el sistema, primero se lanza el servidor en segundo plano o en su propia terminal. Posteriormente, se inicia el cliente pasándole como argumento la dirección IP del equipo donde reside el servidor, utilizando `127.0.0.1` para pruebas locales.

```
./servidor
./cliente 127.0.0.1
```

4.3. Explicacion del codigo del servidor

El servidor se tuvieron que modificar unos archivos, en los cuales, para mantener un control sobre las peticiones del cliente, para mejorar lo que se muestra al iniciarse el servicio, se modificó el archivo de `alumnos_svc.c` agregando rutinas de red de la API POSIX.

```
if (gethostname(host, sizeof(host)) == 0 &&
    getaddrinfo(host, NULL, &hints, &res) == 0 && res != NULL) {
    (void)getnameinfo(res->ai_addr, res->ai_addrlen, ip, sizeof(ip), NULL,
        0,
        NI_NUMERICHOST);
    freeaddrinfo(res);
}

printf("[SISTEMA] Servidor RPC iniciado correctamente. Escuchando
   peticiones en IP %s (Puerto asignado por rpcbind)\n", ip);
fflush(stdout);
```

Esto fue algo sencillo, solamente para que al momento de prender el servidor, el servidor en su terminal muestre un mensaje desde donde nos esta escuchando, esto para evitar poner el comando `hostname -I`.

Para esto el proceso obtiene primero el nombre del server, a través de la función `gethostname`. Con este nombre, para esto despues de hacer las comprobaciones de obtener la ip, para despies, se imprimen estos

datos en la consola para confirmar al administrador que el servicio está activo y disponible en la red local, utilizando `fflush` para vaciar de inmediato el búfer de salida.

Para llevar un registro de los clientes, se implementó una función `log_peticion` en `alumnos_server.c`.

```c
static void log_peticion(struct svc_req *rqstp, const char *operacion){
    char ip[NI_MAXHOST] = "desconocida";
    char puerto[NI_MAXSERV] = "0";

    if (rqstp != NULL && rqstp->rq_xprt != NULL) {
        struct netbuf *caller = &rqstp->rq_xprt->xp_rtaddr;
        if (caller->buf != NULL && caller->len > 0) {
            struct sockaddr_storage ss;
            socklen_t salen;
            size_t copiar;

            memset(&ss, 0, sizeof(ss));
            copiar = caller->len < sizeof(ss) ? caller->len : sizeof(ss);
            memcpy(&ss, caller->buf, copiar);
            salen = (socklen_t)copiar;

            if (getnameinfo((struct sockaddr *)&ss, salen, ip, sizeof(ip),
                puerto,
                sizeof(puerto), NI_NUMERICHOST | NI_NUMERICSERV) != 0) {
                strcpy(ip, "desconocida");
                strcpy(puerto, "0");
            }
        }
    }

    printf("[RECEPCION] Peticion de %s:%s para la operacion %s\n", ip,
        puerto,
        operacion != NULL ? operacion : "desconocida");
    fflush(stdout);
}
```

Esta función pasa los datos de la estructura de la solicitud entrante `rqstp` y valida que existan los punteros asociados y copia los bytes sin procesar contenidos en `xp_rtaddr` hacia `sockaddr_storage` de la máquina local. Con la dirección ya detectectada localmente, la función saca la informacion de el host numérico y el puerto de origen exactos de quien realizó la llamada remota, esto para dejar un registro en la salida cada vez que un servicio es invocado.

Para la parte del registro, para la parte del receptor este procesa primero una chekeo de los datos.

```c
bool_t *
registrar_alumno_1_svc(alumno *argp, struct svc_req *rqstp){
    static bool_t result;
    FILE *f;
    char linea[MAX_LINE];
    alumno aux;

    log_peticion(rqstp, "registrar_alumno");
    result = FALSE;

    if (argp == NULL || campo_vacio(argp->nombre) || campo_vacio(argp->
        apellido) ||
         campo_vacio(argp->curso) || argp->edad < 0) {
         return &result;
    }

    f = fopen(DB_FILE, "a+");
    if (f != NULL) {
         rewind(f);
         while (fgets(linea, sizeof(linea), f) != NULL) {
             memset(&aux, 0, sizeof(aux));
             if (parsear_linea(linea, &aux) && aux.id == argp->id) {
                 liberar_alumno(&aux);
                 fclose(f);
                 return &result;
             }
             liberar_alumno(&aux);
         }
         if (fseek(f, 0, SEEK_END) == 0 && escribir_alumno(f, argp)) {
             result = TRUE;
         }
         fclose(f);
    }
    return &result;
}
```

La función revisa que los atributos del paquete entrante, si detectecte punteros de cadena nulos, debido a que primero como result es FALSE, entonces se detecta que no hay nada aun, en caso de que el cliente ponga algo, al momento de pasar a la siguiente verificacion en el if, entonces, este pasaria result ya no nos serviria, solo sirve como una bandera para indicar si es valido o es invalido el resultado de esta funcion en primer if

se detecta que al menos algo este lleno, entonces en caso de qeu no se entre al if se procede a abrir el origen de datos en modo de editar. Antes a ordenar, el algoritmo rebobina el puntero interno del archivo mediante `rewind` para lograr un escaneo inicial desde el byte base.

Si dentro de esta iteración se detecta un registro donde la clave numérica es la misma exactamente con el identificador propuesto en la orden nueva a agregar del cliente, la función directamente dice que no es posible por que ya se encuentra uno igual.

```
resultado_busqueda *
buscar_alumno_1_svc(busqueda *argp, struct svc_req *rqstp){
    static resultado_busqueda result;
    /* Variables locales ... */

    f = fopen(DB_FILE, "r");
    if (f != NULL) {
        int buscar_por_id = (argp->id > 0);
        while (fgets(linea, sizeof(linea), f) != NULL) {
            memset(&aux, 0, sizeof(aux));
            if (!parsear_linea(linea, &aux)) continue;

            int coincide = 0;
            if (buscar_por_id) {
                if (aux.id == argp->id) coincide = 1;
            } else {
                int match_nom = campo_vacio(argp->nombre) || (strcasecmp(
                    aux.nombre, argp->nombre) == 0);
                int match_cur = campo_vacio(argp->curso) || (strcasecmp(
                    aux.curso, argp->curso) == 0);
                if (match_nom && match_cur) coincide = 1;
            }

            if (coincide) {
                result.status = 0;
                result.resultado_busqueda_u.al = aux;
                fclose(f);
                return &result;
            }
            liberar_alumno(&aux);
        }
        fclose(f);
    }
    return &result;
```

```
    }
```

Al recibir la parte de las estructuras, se crea la funcion, donde con un valor estatico llamado `resultado_busqueda`, esta funcion con la ayuda de la lectura del archivo, que lo abre en modo de lectura, el método clasifica el rastreo de la clave determinando si se puso una clave de id que sea superior a 0, para esto, a lo largo del while, este cadenas extrae las cadenas extraídas de origen del mismo archivo, ahora bien, en la parte de la condicional en el if, se hace uso del parseo de la linea, donde esta vez se utilizan las direcciones de memoria despues de esto, para hacer una busqueda mejor, se revisa el id, revisan que sean los mismos, para esto accedemos al campo id de el tipo de estructura de estudiante, junto con la variable auxiliar esto para levantar la bandera llamda coincide, no obstante, en caso de de cumplir esto, llegamos a la parte del else, donde en contraparte, bajo la falta explícita de este número se aplican evaluaciones ignorando mayúsculas y minúsculas con ayuda de la funcion llamada (`strcasecmp`). Esto hace que al encontrar equivalencias parciales si las variables anteriroes son correctas entonces, la comprobación simultánea es validada como verdaderos, por lo que con esto ya levantamos la bandera.

Ahora la ultima parte tenemos la ultima condicional, donde lo hacemos para estar libre de fallos, utilizamos la variable (`status` en 0) esto hace que el servidor recargue a la variable completa extraída dentro del .txt, asi de igual forma se libera la memoria para que no siga consumiendo.

En la siguientes modificaciones fue en la parte de registros para las eliminaciones, el diseño que se nos genero por el .x, se reescribrió esto para hacer modificaciones de forma atómica en los registros de memoria.

```
    bool_t *
    actualizar_alumno_1_svc(alumno *argp, struct svc_req *rqstp){

        static bool_t result;
        FILE *f, *ftmp;
        /* ... */

        f = fopen(DB_FILE, "r");
        ftmp = fopen(TMP_DB_FILE, "w");

        while (fgets(linea, sizeof(linea), f) != NULL) {
            memset(&aux, 0, sizeof(aux));
            parsear_linea(linea, &aux);

            if (aux.id == argp->id) {
                if (!campo_vacio(argp->nombre)) { free(aux.nombre); aux.nombre
                    = strdup(argp->nombre); }
                if (!campo_vacio(argp->apellido)) { free(aux.apellido); aux.
                    apellido = strdup(argp->apellido); }
```

```
                if (!campo_vacio(argp->curso)) { free(aux.curso); aux.curso =
                    strdup(argp->curso); }
                if (argp->edad >= 0) { aux.edad = argp->edad; }
                result = TRUE;
            }
            escribir_alumno(ftmp, &aux);
            liberar_alumno(&aux);
        }
        fclose(f);
        fclose(ftmp);

        if (result) {
            remove(DB_FILE);
            rename(TMP_DB_FILE, DB_FILE);
        } else {
            remove(TMP_DB_FILE);
        }
        return &result;
    }
```

En la parte operativa de este codigo, comenzamos con la definicion de un txt temporal donde ese es el que vamos a modificar por que lo abriremos en modo de escritura, el original solo sera abierto en modo de lectura, despues de esto utiliznado fgets, volvemos a leer las lineas de los archivos, donde en seguida en tanto sean iguales secuencialmente los datos evaluados, que justo esto se hace en la condicional con el id, ahora por medio de comparaciones de las estructuras en donde el nuevo nodo se inspecciona con respecto la nueva de información, la memoria dinámica que teniamos antes reservadda, ahora sera liberada. De este modo, si la comparacion de las cadenas es true, el codigo avanza desocupado dejando la parte de la matriz del cliente originario. En caso de tener un resultado false, entonces el servicio elimina el bloque original para seguir los siguientes datos numéricos.

4.4. Desarrollo del Cliente

En el código principal del cliente es en el codigo de (`alumnos_client.c`), la parte inicial es verificando la disponibilidad de conexión mediante el método `clnt_create`, al cual se le proporciona la IP enviada por la consola, el nombre del programa, la versión y el protocolo de red que en este caso es el de TCP. Una vez establecida la comunicacion, inicia el ciclo que inicia toda la comunicacion del mismo.

Para prevenir búsquedas, actualizaciones o eliminaciones que no tienen sentido sin información existente en el servidor, se forzó la llamada a `listar_todos` en los casos del menú principal, eto para tener una mejor visualizacion.

```c
    case 3:
        if (listar_todos(clnt) <= 0) {
            break;
        }
        printf("\nDeja un campo vacio y presiona Enter para conservar su valor
            actual.\n");
        al.id = leer_entero("ID del alumno a actualizar: ");
        leer_linea("Nuevo nombre: ", nombre, sizeof(nombre));
        leer_linea("Nuevo apellido: ", apellido, sizeof(apellido));
        al.edad = leer_entero_opcional("Nueva edad (Enter para conservar): ",
            -1);
```

La función de listado solicita todos los registros a través de XDR y retorna un código numérico. Al validar el resultado, si el código determina que no hay alumnos (por listar un apuntador nulo), el condicional invoca la instrucción `break` forzando el cierre. El resultado retorna al usuario nuevamente el menú, bloqueando el avance.

Si la lista de valores sí contiene información, se sigue con un bloque de repeticion, donde la lectura de argumentos que se ponen desde consola se hizo con funciones diseñadas como lo son las de `leer_linea` o `leer_entero_opcional`. Si el nosotros le emtemos directamente un `Enter` al solicitar el texto, la cadena pasa de se registra vacía, lo cambiamos por un número preconfigurado que este mismo actua como bandera, donde lo ponemos posicionado en `-1`. Si cacha el marcador `-1` o de caracteres en los apuntadores serializados de cadena, salta asíncronamente su actualización y pasa por retener en el registro la información que se obtuvo.

4.5. Resultados:

4.5.1. Opcion 1 (Registro de alumnos):

Para la primera parte, tenemos al inicio del servidor, donde al momento de inicair el servidor el servidor manda un mensaje en la consola, indicando que ip es la que esta usando y cual fue la asignada por rpcbind, ahora bien, una vez iniciando el servidor, los demas clientes se pueden conectar, lanzandoles el menu principal, donde el cliente 1, lo primero que hace es registrar un alumno, donde se le pide que se le asigne el ID, el nombre, donde sera el nombre de el integrante Adolfo de jesus, despues de esto se le pide la edad y el curso, por ultimo el servidor, le responde con un mensaje de cofnirmacion y el servidor en la terminal y en txt, se actuliza correctamente y al mismo tiempo.

Despues de esto, el cliente 2, tambien registrara un alumno, donde este mismo se le pediran los mismos datos y se actulziara de la misma forma, donde en la terminal del servidor, se ve como la ip cambia entre dispositivos y se ve como se actuliza correctame el txt.

Ahora bien, en el cliente 1, que tiene la direccion ip `172.27.61.105` con el puerto `51880` y el cliente 2, tiene la ip `172.27.61.64` con el puerto `45828`. Estos dos clientes, seguiran registratnado otros dos alumnos, donde quedara de la siguiente manera y se vera asi tanto en la terminal de los clientes:

4.5.2. Opcion 2 (Buscar alumno):

Los clientes biscaran los alumnos, ya sea por ID, nombre, o algun campo relacionado, donde al cliente se le traera el alumno con sus atributos, pero en el servidor, se vera que usuario hizo la solicitud en la terminal.

4.5.3. Opcion 3 (Actulizacion de alumno):

Para la parte de actulizacion de un alumno, lo que se hace principamente, es listar todos los alumnos registrados en la terminla, una vezz que lo localice, el cliente lo buscara por el ID, donde, se podran actulizar todos los campos, pero en caso de no querer actulizar un campo, se puede dejar vacio dandole un enter y se quedara como esta, despues de esto en el setvidor se veran dos llamdas o dos solicituds, una que es listar alumnos, y otro que es la funcion de actulizar alumno, esto se ve en la terminal como se hacen las dos solicitudes se ven en la misma ip, ademas de eso, se ve como se actuliza correctaente el txt, a comparacion del anterior:

4.6. Opcion 4 (Eliminar alumno:)

Para eliminar un alumno, en este caso se le muestra la lista de los alumnos a el cliente y solo se le pide el ID que se desee eliminar y este mismo se eliminara autoamticamente tanto del registro como del txt para tener un correcto control del mismo.

5. Actividad 2

5.1. Planteamiento de la Solución

La solución propuesta consiste en un sistema distribuido basado en el paradigma de **Llamada a Procedimiento Remoto (RPC)**, utilizando el framework **gRPC**. El objetivo es manejar un registro de alumnos permitiendo operaciones de creación, consulta, actualización, eliminación y listado (CRUD).

Se decidió guardar los datos en un archivo de texto usando formato **JSON**. Esto es más sencillo que usar una base de datos grande y complicada, y permite que el programa guarde y recupere la información fácilmente cada vez que se ejecuta, además de que es fácil de mover y mantener.

5.2. Diseño y Justificación del Archivo `.proto`

El archivo `alumno.proto` funciona como un acuerdo claro entre el cliente y el servidor. Ahí se define qué datos se van a enviar y recibir, y qué acciones puede hacer el servidor. Lo importante es que estas reglas no dependen del lenguaje de programación, así que ambos lados pueden entenderse sin problema. A continuación se muestra el código base del protocolo:

```
//Version 3
syntax = "proto3";

package alumnos;

// Funciones del servidor
service AlumnoService {
  // Operaciones unarias (Peticion-Respuesta)
  rpc Registrar (RegistrarRequest) returns (RegistrarResponse);
  rpc Consultar (ConsultarRequest) returns (Alumno);
  rpc Actualizar (ActualizarRequest) returns (Respuesta);
  rpc Eliminar (EliminarRequest) returns (Respuesta);

  // Operaciones de Server Streaming
  rpc BuscarPorCriterio (BuscarRequest) returns (stream Alumno);
  rpc ListarTodos (ListarRequest) returns (stream Alumno);
}

// MENSAJES

// Estructura principal de un alumno.
message Alumno {
  string id = 1;
  string nombre = 2;
  string apellido = 3;
  int32 edad = 4;
  string curso = 5;
  string direccion = 6;
}

message RegistrarRequest {
  string nombre = 1;
  string apellido = 2;
  int32 edad = 3;
  string curso = 4;
```

```
  string direccion = 5;
}

message RegistrarResponse {
  bool exito = 1;
  string mensaje = 2;
  string id = 3;
}

message ConsultarRequest {
  string id = 1;
}

message ActualizarRequest {
  string id = 1;
  string nombre = 2;
  string apellido = 3;
  int32 edad = 4;
  string curso = 5;
  string direccion = 6;
}

message EliminarRequest {
  string id = 1;
}

// Respuesta generica de exito/fallo
message Respuesta {
  bool exito = 1;
  string mensaje = 2;
}

message BuscarRequest {
  string campo = 1;      // Puede ser "nombre", "apellido" o "curso"
  string criterio = 2;   // Lo que el usuario escribio para buscar
}

// Mensaje vacio para listar todos
message ListarRequest {
}
```

Análisis y Justificación del Diseño:

El diseño de este archivo `.proto` sigue buenas prácticas para crear APIs con gRPC, pero se puede entender de forma sencilla así:

- **Definición de Servicios (`AlumnoService`):** Se organizaron las funciones del servidor en dos tipos:
  - *Operaciones Unarias:* Son las más simples. El cliente hace una petición y el servidor responde una sola vez (por ejemplo: registrar, consultar, actualizar o eliminar).
  - *Operaciones de Streaming (`stream Alumno`):* En algunos casos, el servidor no envía todo de golpe, sino que manda los datos poco a poco. Esto evita usar demasiada memoria y hace que funcione mejor cuando hay muchos registros.
- **Patrón de Mensajes Request/Response:** En lugar de usar el mismo tipo de dato para todo, se crearon mensajes específicos para cada acción (como `RegistrarRequest` o `EliminarRequest`). Esto permite que en el futuro se puedan agregar nuevos datos sin afectar lo que ya funciona.
- **Tipado Fuerte y Ligereza:** Se usan tipos de datos simples y definidos (como números, texto o verdadero/falso). Además, gRPC convierte esta información a un formato binario, lo que hace que los datos se envíen más rápido y ocupen menos espacio que en otras opciones como REST.

Análisis de la Estructura de Datos y Funciones

Para cumplir con el criterio de una definición clara del servicio, los mensajes y los métodos, a continuación se desglosa la estructura central de información y el catálogo de operaciones, complementados con sus respectivos fragmentos de código para ilustrar su funcionamiento.

5.2.1. La Estructura Principal: `Alumno` (Mensajes)

El mensaje `Alumno` es la entidad central (el modelo de datos) de todo el sistema. Define exactamente qué piezas de información conforman a un estudiante dentro de la base de datos:

```
// Estructura principal de un alumno.
message Alumno {
  string id = 1;
  string nombre = 2;
  string apellido = 3;
  int32 edad = 4;
  string curso = 5;
  string direccion = 6;
}
```

- **`id` (string):** Un identificador único (generado aleatoriamente por el servidor) para distinguir a cada alumno, incluso si tienen el mismo nombre.
- **`nombre` y `apellido` (string):** Cadenas de texto para la identidad del estudiante.
- **`edad` (int32):** Se utiliza un entero de 32 bits, que es un tipo de dato numérico altamente optimizado en gRPC, ideal para operaciones lógicas.
- **`curso` y `direccion` (string):** Cadenas de texto que almacenan información académica y de localización.

5.2.2. Las Funciones del Servidor (Servicio y Métodos RPC)

El servicio expone 6 métodos principales, los cuales cubren el ciclo de vida completo de los datos (CRUD) y consultas avanzadas. A continuación se explica su lógica interna de lado del servidor:

1. Registrar y Consultar (Operaciones Básicas)

```python
def Registrar(self, request, context):
    nuevo_id = str(uuid.uuid4())[:8] # Generacion de ID unico
    db = leer_json()
    db[nuevo_id] = {
        "id": nuevo_id, "nombre": request.nombre,
        "apellido": request.apellido, "edad": request.edad,
        "curso": request.curso, "direccion": request.direccion
    }
    guardar_json(db)
    return alumno_pb2.RegistrarResponse(exito=True, mensaje="Alumno registrado
        .", id=nuevo_id)

def Consultar(self, request, context):
    db = leer_json()
    if request.id in db:
        return alumno_pb2.Alumno(**db[request.id])
    context.set_code(grpc.StatusCode.NOT_FOUND) # Manejo de error
    return alumno_pb2.Alumno()
```

- **`Registrar`:** Recibe los datos crudos del estudiante. El servidor le genera un ID único mediante la librería `uuid`, lo guarda en el diccionario de la base de datos y devuelve un mensaje de confirmación con el ID asignado.
- **`Consultar`:** Recibe únicamente el ID. Si lo encuentra en el archivo JSON, devuelve la estructura `Alumno` completa. Si no, notifica al cliente con un código de error `NOT_FOUND`.

2. Actualizar y Eliminar (Modificación de Estado)

```
def Actualizar(self, request, context):
    db = leer_json()
    if request.id in db:
        db[request.id] = { # Sobrescritura del registro
            "id": request.id, "nombre": request.nombre,
            "apellido": request.apellido, "edad": request.edad,
            "curso": request.curso, "direccion": request.direccion
        }
        guardar_json(db)
        return alumno_pb2.Respuesta(exito=True, mensaje="Alumno actualizado.")
    return alumno_pb2.Respuesta(exito=False, mensaje="El ID no existe.")

def Eliminar(self, request, context):
    db = leer_json()
    if request.id in db:
        del db[request.id] # Borrado atomico de la llave en el diccionario
        guardar_json(db)
        return alumno_pb2.Respuesta(exito=True, mensaje="Alumno eliminado.")
```

- **`Actualizar`:** Recibe el ID del alumno y sus nuevos datos. Sobrescribe la información antigua directamente en la memoria y guarda los cambios en el archivo persistente.
- **`Eliminar`:** Busca el ID y utiliza la instrucción `del` para remover todo el bloque de información asociado a ese estudiante en la base de datos.

3. Buscar y Listar (Operaciones de Streaming)

```
def BuscarPorCriterio(self, request, context):
    db = leer_json()
    criterio = request.criterio.lower()
    campo = request.campo.lower()
    for alumno_data in db.values():
        if campo in alumno_data and criterio in str(alumno_data[campo]).lower
            ():
            yield alumno_pb2.Alumno(**alumno_data) # Retorno en flujo (Stream)

def ListarTodos(self, request, context):
    db = leer_json()
    for alumno_data in db.values():
        yield alumno_pb2.Alumno(**alumno_data) # Retorno en flujo (Stream)
```

- **`BuscarPorCriterio`:** Es una función flexible que aplica un filtro ignorando mayúsculas y minúsculas (`lower()`). Utiliza la instrucción `yield` para transmitir en flujo continuo (*stream*) todos los alumnos que coincidan con la búsqueda.
- **`ListarTodos`:** Lee el archivo JSON completo y comienza a enviar, uno por uno y sin interrupciones, todos los registros al cliente mediante un flujo optimizado.

5.3. Desarrollo del Lado del Servidor

¿Qué se pretende hacer en el lado del Servidor?

El servidor tiene como propósito fundamental actuar como el núcleo (backend) del sistema. Sus objetivos principales son:

1. **Exponer los servicios** definidos en el contrato `.proto`, escuchando peticiones en un puerto de red específico de forma ininterrumpida.
2. **Garantizar la integridad de los datos en entornos concurrentes**. Dado que gRPC permite conexiones simultáneas, el servidor pretende manejar múltiples clientes al mismo tiempo sin que la escritura en el archivo JSON (`alumnos.json`) sufra colisiones o se corrompa.

Para lograr esto, la infraestructura del servidor se implementó de la siguiente manera:

5.3.1. 1. Concurrencia y Persistencia Segura (I/O)

```python
import threading
import json
import os

JSON_FILE = "alumnos.json"
_lock = threading.Lock() # Bloqueo para evitar colisiones

def guardar_json(datos):
    """Guarda un diccionario de Python en el archivo JSON."""
    with _lock: # Solo un hilo a la vez puede entrar aqui
        with open(JSON_FILE, "w") as f:
            json.dump(datos, f, indent=4)
```

Explicación Detallada del Bloque:

- **Manejo del Archivo (`JSON_FILE`):** Se utiliza un archivo de texto plano estandarizado para persistir la información sin requerir un motor de base de datos pesado.

- **Prevención de Condición de Carrera (`threading.Lock()`):** En un entorno distribuido, múltiples clientes pueden intentar modificar datos al mismo tiempo. Si el Cliente A y el Cliente B intentan registrar un alumno en el mismo milisegundo, el archivo `alumnos.json` podría corromperse. La variable `_lock` actúa como un semáforo (Mutex).
- **Bloque de Exclusión Mutua (`with _lock`):** Esta instrucción garantiza la seguridad en la escritura. Si un hilo (petición de un cliente) entra a este bloque, bloquea la puerta. Cualquier otra petición que intente guardar datos quedará en pausa hasta que el primer hilo termine, manteniendo la integridad absoluta de la base de datos.

5.3.2. 2. Lógica de Negocio (Clase `AlumnoServidor`)

```python
import uuid
import grpc
import alumno_pb2, alumno_pb2_grpc

class AlumnoServidor(alumno_pb2_grpc.AlumnoServiceServicer):
    def Registrar(self, request, context):
        nuevo_id = str(uuid.uuid4())[:8] # Generacion de ID corto
        db = leer_json()
        db[nuevo_id] = {
            "id": nuevo_id, "nombre": request.nombre,
            "apellido": request.apellido, "edad": request.edad,
            "curso": request.curso, "direccion": request.direccion
        }
        guardar_json(db)
        return alumno_pb2.RegistrarResponse(exito=True, mensaje="Alumno
            registrado.", id=nuevo_id)

    def BuscarPorCriterio(self, request, context):
        db = leer_json()
        criterio = request.criterio.lower()
        campo = request.campo.lower()
        for alumno_data in db.values():
            if campo in alumno_data and criterio in str(alumno_data[campo]).
                lower():
                yield alumno_pb2.Alumno(**alumno_data) # Retorno en flujo (
                    Stream)
```

Explicación Detallada de las Funciones: La clase hereda de la plantilla *Servicer* generada automáticamente por gRPC.

- **Función `Registrar` (Petición Unaria):**

  1. *Generación de Identidad:* Utiliza la librería `uuid.uuid4()` para generar un identificador criptográficamente único, recortándolo a 8 caracteres (`[:8]`) para que sea más amigable para el usuario.

  2. *Mapeo de Datos:* Descarga la base de datos a un diccionario en memoria (`db`), crea una nueva entrada usando el ID como llave, y le asigna los datos desempaquetados de la petición del cliente (`request.nombre`, etc.).

  3. *Respuesta:* Llama a la función segura `guardar_json()` y retorna un mensaje `RegistrarResponse` confirmando el éxito de la operación.

- **Función `BuscarPorCriterio` (Server Streaming):**

  1. *Normalización:* Convierte tanto el campo objetivo como la palabra clave a minúsculas (`.lower()`) para realizar búsquedas insensibles a mayúsculas, mejorando la experiencia del usuario.

  2. *Transmisión Continua (`yield`):* A diferencia de funciones tradicionales que usan `return` para devolver un solo bloque de datos, esta función utiliza `yield`. Esto la convierte en un **generador**. Cada vez que el ciclo `for` encuentra una coincidencia, la emite inmediatamente por el canal gRPC hacia el cliente sin cerrar la conexión. Esto evita saturar la memoria RAM del servidor al no tener que guardar listas gigantes antes de enviarlas.

5.3.3. 3. Inicialización Multi-hilo

```python
from concurrent import futures

def iniciar_servidor():
    # Servidor capaz de manejar 10 clientes al mismo tiempo
    servidor = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    alumno_pb2_grpc.add_AlumnoServiceServicer_to_server(AlumnoServidor(),
        servidor)
    servidor.add_insecure_port("[::]:50051")
    servidor.start()
    servidor.wait_for_termination()
```

Explicación Detallada de la Inicialización:

- **El Pool de Hilos (`ThreadPoolExecutor(max_workers=10)`):** gRPC está diseñado para alta concurrencia. En lugar de procesar peticiones en una fila lenta, el servidor crea una "piscina"de 10 trabajadores

independientes. Esto le otorga la capacidad de atender y resolver hasta 10 llamadas de red simultáneas en paralelo.

- **El Puerto y Canal (`add_insecure_port("[::]:50051")`):** La directiva `[::]` le indica al servidor que escuche tráfico en todas las direcciones IPv4 e IPv6 disponibles en la máquina local. Al llamarse *insecure*, indica que la conexión es de texto plano sin certificados SSL/TLS, lo cual es ideal para fases de desarrollo.
- **Ciclo de Vida (`wait_for_termination()`):** Es una función bloqueante crítica. Si no estuviera, el script de Python pasaría por la línea `start()` e inmediatamente terminaría la ejecución, apagando el servidor en un segundo. Esta instrucción mantiene el hilo principal abierto e iterando de forma infinita hasta que se detenga el proceso manualmente (ej. mediante `Ctrl+C`).

5.4. Desarrollo del Lado del Cliente

¿Qué se pretende hacer en el lado del Cliente?

El cliente es la parte que interactúa directamente con el usuario. Sus objetivos son:

1. **Ocultar la complejidad de la red:** Para el usuario, todo debe sentirse como un programa normal en su computadora (con un menú en consola). El cliente se encarga de enviar las peticiones al servidor y de mostrar las respuestas sin que el usuario note lo que pasa detrás.
2. **Manejo de errores:** Si ocurre un problema en la red o en el servidor, el cliente debe manejarlo correctamente y mostrar un mensaje claro, en lugar de cerrarse de forma inesperada.

Para lograr esto, el cliente se estructuró de la siguiente manera:

5.4.1. 1. Conexión Dinámica y Canal gRPC

```python
import grpc
import alumno_pb2, alumno_pb2_grpc
import sys

def ejecutar(ip_servidor):
    direccion = f"{ip_servidor}:50051"
    # Abrimos el canal de comunicacion hacia la IP indicada
    with grpc.insecure_channel(direccion) as channel:
        # El stub actua como un puente transparente hacia el servidor
        stub = alumno_pb2_grpc.AlumnoServiceStub(channel)
        # ... Inicio del bucle del menu ...
```

Explicación Detallada del Bloque:

- **Flexibilidad de Red (`ip_servidor`):** En lugar de quemar (*hardcodear*) la IP de conexión en el código, el script está diseñado para recibir la dirección del servidor como parámetro (usualmente mediante `sys.argv` al ejecutar desde la terminal). Esto permite desplegar el cliente en cualquier máquina de la red sin modificar el código fuente.

- **Gestión de Conexión (`grpc.insecure_channel`):** Se establece una tubería de red directa mediante el protocolo HTTP/2 (el estándar subyacente de gRPC). La cláusula `with` es una medida de seguridad (context manager) que asegura que, sin importar si el programa termina correctamente o falla por un error, el canal de red se cerrará de forma limpia, liberando el puerto.

- **El Patrón Stub (`AlumnoServiceStub`):** El *Stub* es un proxy o representante local. Para el programador del cliente, `stub` parece un objeto de Python normal con funciones normales, pero internamente, es el encargado de interceptar esas llamadas, serializar los datos a código binario y enviarlos a través del canal de red hacia el servidor real.

5.4.2. 2. Empaquetado y Robustez (Manejo de Errores)

```python
            elif opcion == "1":
                n = input("Nombre: ")
                # ... (captura de datos extra omitida por brevedad) ...

                # Empaquetamos en el Request correspondiente
                request = alumno_pb2.RegistrarRequest(nombre=n, apellido="X",
                    edad=20, curso="Y", direccion="Z")
                try:
                    # Invocacion del procedimiento remoto
                    respuesta = stub.Registrar(request)
                    if respuesta.exito:
                        print(f"ID GENERADO: {respuesta.id}")
                except grpc.RpcError as err:
                    # Captura de fallos de red o caidas del servidor
                    print(f"\n[!] Error de conexion: {err.details()}")
```

Explicación Detallada de las Peticiones:

- **Construcción del Mensaje (`alumno_pb2.RegistrarRequest`):** Antes de enviar cualquier dato, este debe encapsularse estrictamente en el formato definido en el `.proto`. gRPC valida fuertemente los tipos de datos en este punto (ej. no permite enviar un *string* si el proto definió un *int32*).

- **Invocación Síncrona (`stub.Registrar(request)`):** Al ejecutar esta línea, el hilo del cliente se bloquea temporalmente esperando la respuesta del servidor.

- **Tolerancia a Fallos (`try-except grpc.RpcError`):** Esta es quizás la sección más crítica en un sistema distribuido. Si el servidor se apaga repentinamente, si el cable de red se desconecta, o si la base de datos interna falla, la librería gRPC arrojará un error de tipo `RpcError`. Al capturarlo aquí, se evita que todo el script de Python del cliente se cierre abruptamente. En su lugar, se imprime un mensaje de error detallado (`err.details()`) y el usuario es retornado al menú principal de forma segura.

5.4.3. 3. Consumo de Streaming de Datos

```
            elif opcion == "5":
                campo = input("Campo a buscar: ")
                criterio = input("Que deseas buscar?: ")
                request = alumno_pb2.BuscarRequest(campo=campo, criterio=
                    criterio)

                print("\n--- Resultados ---")
                # Iteracion sobre el stream de datos proveniente del servidor
                for alumno in stub.BuscarPorCriterio(request):
                    mostrar_alumno(alumno)
```

5.5. Resultados de la Actividad 2

5.5.1. Local

Se mostrara a continuación la ejecución del servidor y el cliente en la misma máquina, utilizando localhost como dirección de conexión.

```
PS C:\Users\Adolfo\Distribuidos\project\Actividad2> python servidor.py
Servidor gRPC iniciado en el puerto 50051...
```

```
PS C:\Users\Adolfo\Distribuidos\project\Actividad2> python cliente.py

[ Intentando conectar con el servidor en: localhost ]

--- MENÚ DE ALUMNOS ---
1. Registrar Alumno
2. Consultar Alumno
3. Actualizar Alumno
4. Eliminar Alumno
5. Buscar por criterio
6. Listar todos
0. Salir
Elige una opción:
```

Figura 5: Diagrama de conexión entre el Cliente y el Servidor gRPC.

Opción 1: Registrar Alumno

```
PS C:\Users\Adolfo\Distribuidos\project\Actividad2> python servidor.py
Servidor gRPC iniciado en el puerto 50051...
```

```
--- MENÚ DE ALUMNOS ---
1. Registrar Alumno
2. Consultar Alumno
3. Actualizar Alumno
4. Eliminar Alumno
5. Buscar por criterio
6. Listar todos
0. Salir
Elige una opción: 1
Nombre: Angel Husiel
Apellido: Lara Hernandez
Edad: 21
Curso: Sistemas Distribuidos
Dirección: Avenida Insurgentes Sur 742, Colonia Del Valle Norte, Alcaldía Benito Juárez, C.P. 03103, Ciudad de México, México.

  ========================================
  Alumno registrado correctamente.
   ID GENERADO: 21a4b088
  ========================================
  Este id fue generado automaticamente.

Alumno registrado correctamente. ID asignado: 21a4b088
```

Figura 6: Opción 1: Registrar Alumno.

```
project > Actividad2 > {} alumnos.json > ...
1  {
2      "77aa599c": {
3          "id": "77aa599c",
4          "nombre": "Adolfo",
5          "apellido": "Garcia",
6          "edad": 21,
7          "curso": "Distribuidos",
8          "direccion": "Iztapalapa"
9      },
10     "b99eb83d": {
11         "id": "b99eb83d",
12         "nombre": "Paula ",
13         "apellido": "Garcia",
14         "edad": 21,
15         "curso": "Online",
16         "direccion": "Iztapalapa"
17     },
18     "21a4b088": {
19         "id": "21a4b088",
20         "nombre": "Angel Husiel",
21         "apellido": "Lara Hernandez",
22         "edad": 21,
23         "curso": "Sistemas Distribuidos",
24         "direccion": "Avenida Insurgentes Sur 742, Colonia Del Valle Norte, Alcald\u00eda Benito
25     }
26 }
```

Figura 7: Cambios en el archivo JSON de la Opción 1.

Opción 2: Consultar Alumno

```
PS C:\Users\Adolfo\Distribuidos\project\Actividad2> python servidor.py
Servidor gRPC iniciado en el puerto 50051...
[]
```

```
Alumno registrado correctamente. ID asignado: 21a4b088

--- MENÚ DE ALUMNOS ---
1. Registrar Alumno
2. Consultar Alumno
3. Actualizar Alumno
4. Eliminar Alumno
5. Buscar por criterio
6. Listar todos
0. Salir
Elige una opción: 2
ID del alumno: 21a4b088

--- Resultado ---
ID: 21a4b088 | Angel Husiel Lara Hernandez | Edad: 21 | Curso: Sistemas Distribuidos | Dir: Avenida Insurgentes Sur 742, Colonia Del Valle
 Norte, Alcaldía Benito Juárez, C.P. 03103, Ciudad de México, México.
```

Figura 8: Opción 2: Consultar Alumno.

Opción 3: Actualizar Alumno

```
PS C:\Users\Adolfo\Distribuidos\project\Actividad2> python servidor.py
Servidor gRPC iniciado en el puerto 50051...
[]
```

```
--- Resultado ---
ID: 21a4b088 | Angel Husiel Lara Hernandez | Edad: 21 | Curso: Sistemas Distribuidos | Dir: Avenida Insurgentes Sur 742, Colonia Del Valle
 Norte, Alcaldía Benito Juárez, C.P. 03103, Ciudad de México, México.

--- MENÚ DE ALUMNOS ---
1. Registrar Alumno
2. Consultar Alumno
3. Actualizar Alumno
4. Eliminar Alumno
5. Buscar por criterio
6. Listar todos
0. Salir
Elige una opción: 3

  [ ACTUALIZAR ALUMNO ]
  ID del alumno a actualizar: 21a4b088

  Alumno encontrado: Angel Husiel Lara Hernandez

  ¿Qué atributo deseas modificar?
  1. Nombre     (Actual: Angel Husiel)
  2. Apellido   (Actual: Lara Hernandez)
  3. Edad       (Actual: 21)
  4. Curso      (Actual: Sistemas Distribuidos)
  5. Dirección  (Actual: Avenida Insurgentes Sur 742, Colonia Del Valle Norte, Alcaldía Benito Juárez, C.P. 03103, Ciudad de México, Méxi
o.)
  6. Terminar y GUARDAR cambios
  0. Cancelar actualización
  Elige una opción: 2
  Nuevo Apellido: Lara Saucedo

  ¿Qué atributo deseas modificar?
  1. Nombre     (Actual: Angel Husiel)
  2. Apellido   (Actual: Lara Saucedo)
  3. Edad       (Actual: 21)
  4. Curso      (Actual: Sistemas Distribuidos)
  5. Dirección  (Actual: Avenida Insurgentes Sur 742, Colonia Del Valle Norte, Alcaldía Benito Juárez, C.P. 03103, Ciudad de México, Méxi
o.)
  6. Terminar y GUARDAR cambios
  0. Cancelar actualización
  Elige una opción: 6

  Alumno actualizado.
```

Figura 9: Opción 3: Actualizar Alumno.

```
main.tex 7, M ●   {} alumnos.json M ×
project > Actividad2 > {} alumnos.json > {} 21a4b088 > abc direccion
2     "77aa599c": {
5         "apellido": "Garcia",
6         "edad": 21,
7         "curso": "Distribuidos",
8         "direccion": "Iztapalapa"
9     },
10    "b99eb83d": {
11        "id": "b99eb83d",
12        "nombre": "Paula ",
13        "apellido": "Garcia",
14        "edad": 21,
15        "curso": "Online",
16        "direccion": "Iztapalapa"
17    },
18    "21a4b088": {
19        "id": "21a4b088",
20        "nombre": "Angel Husiel",
21        "apellido": "Lara Saucedo",
22        "edad": 21,
23        "curso": "Sistemas Distribuidos",
24        "direccion": "Avenida Insurgentes Sur 742, Colonia Del Valle Norte, Alcald\u00eda Benito
25    }
26 }
```

Figura 10: Cambios en el archivo JSON de la Opción 3.

Opción 4: Eliminar Alumno

```
PROBLEMS 17   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SQL HISTORY   TASK MONITOR

PS C:\Users\Adolfo\Distribuidos\project\Actividad2> python servidor.py
Servidor gRPC iniciado en el puerto 50051...

  Alumno actualizado.

--- MENÚ DE ALUMNOS ---
1. Registrar Alumno
2. Consultar Alumno
3. Actualizar Alumno
4. Eliminar Alumno
5. Buscar por criterio
6. Listar todos
0. Salir
Elige una opción: 4
ID del alumno a eliminar: 21a4b088

Alumno eliminado.
```

Figura 11: Opción 4: Eliminar Alumno.

```
main.tex 9, M   4.png   alumnos.json
project > Actividad2 > alumnos.json > ...
1  {
2      "77aa599c": {
3          "id": "77aa599c",
4          "nombre": "Adolfo",
5          "apellido": "Garcia",
6          "edad": 21,
7          "curso": "Distribuidos",
8          "direccion": "Iztapalapa"
9      },
10     "b99eb83d": {
11         "id": "b99eb83d",
12         "nombre": "Paula ",
13         "apellido": "Garcia",
14         "edad": 21,
15         "curso": "Online",
16         "direccion": "Iztapalapa"
17     }
18 }
```

Figura 12: Cambios en el archivo JSON de la Opción 4.

Opción 5: Buscar por Criterio

```
PROBLEMS 21   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SQL HISTORY   TASK MONITOR

PS C:\Users\Adolfo\Distribuidos\project\Actividad2> python servidor.py
Servidor gRPC iniciado en el puerto 50051...
[]

ID del alumno a eliminar: 21a4b088

Alumno eliminado.

--- MENÚ DE ALUMNOS ---
1. Registrar Alumno
2. Consultar Alumno
3. Actualizar Alumno
4. Eliminar Alumno
5. Buscar por criterio
6. Listar todos
0. Salir
Elige una opción: 5
Campo a buscar (id, nombre, apellido, curso, Dir): nombre
Qué deseas buscar?: Adolfo

--- Resultados ---
ID: 77aa599c | Adolfo Garcia | Edad: 21 | Curso: Distribuidos | Dir: Iztapalapa
```

Figura 13: Opción 5: Buscar por Criterio.

Opción 6: Listar Todos los Alumnos

```
PROBLEMS 21   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SQL HISTORY   TASK MONITOR

PS C:\Users\Adolfo\Distribuidos\project\Actividad2> python servidor.py
Servidor gRPC iniciado en el puerto 50051...
[]

--- MENÚ DE ALUMNOS ---
1. Registrar Alumno
2. Consultar Alumno
3. Actualizar Alumno
4. Eliminar Alumno
5. Buscar por criterio
6. Listar todos
0. Salir
Elige una opción: 6

--- Lista de Alumnos ---
ID: 77aa599c | Adolfo Garcia | Edad: 21 | Curso: Distribuidos | Dir: Iztapalapa
ID: b99eb83d | Paula  Garcia | Edad: 21 | Curso: Online | Dir: Iztapalapa
```

Figura 14: Opción 6: Listar Todos los Alumnos.

5.5.2. Remoto

Se mostrara a continuación la ejecución del servidor y el cliente en máquinas diferentes, utilizando la dirección IP del servidor para establecer la conexión.

```
Adaptador de LAN inalámbrica Wi-Fi:

   Sufijo DNS específico para la conexión. . : izzigo.tv
   Dirección IPv6 . . . . . . . . . . . : 2806:2a0:404:8f07:7abe:7420:31e0:5e77
   Dirección IPv6 . . . . . . . . . . . : 2806:2a0:404:8f07:9692:446a:6bb7:4807
   Dirección IPv6 temporal. . . . . . . : 2806:2a0:404:8f07:1c65:52d9:f10b:7f91
   Vínculo: dirección IPv6 local. . . . : fe80::1d3c:e54:7069:9915%13
   Dirección IPv4. . . . . . . . . . . . . . : 192.168.0.252
   Máscara de subred . . . . . . . . . . . . : 255.255.255.0
   Puerta de enlace predeterminada . . . . . : fe80::10:18ff:fe75:587c%13
```

Figura 15: Ip del Servidor.

En este caso la ip del servidor es `192.168.0.252`, por lo que al ejecutar el cliente, se le pasa esta dirección como argumento para establecer la conexión.

```
PROBLEMS 25   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SQL HISTORY   TASK MONITOR

PS C:\Users\Adolfo\Distribuidos\project\Actividad2>   python servidor.py
Servidor gRPC iniciado en el puerto 50051...
```

Figura 16: Servidor Remoto.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                    python - Actividad2

PS C:\Users\adolf\Downloads\Actividad2\Actividad2> python cliente.py 192.168.0.252

 [ Intentando conectar con el servidor en: 192.168.0.252 ]

 --- MENÚ DE ALUMNOS ---
 1. Registrar Alumno
 2. Consultar Alumno
 3. Actualizar Alumno
 4. Eliminar Alumno
 5. Buscar por criterio
 6. Listar todos
 0. Salir
 Elige una opción:

Ln 18, Col 9   Spaces: 4   UTF-8   CRLF   { } Python   Python 3.11 (64-bit)
```

Figura 17: Cliente Remoto.

En el caso de la manera remoto, solo se mostrara las funciones de modificación, ya que en realidad todo lo demás es lo mismo, sol es para reflejar que los cambios también se guardan en el archivo JSON del servidor, aunque el cliente se ejecute en otra máquina.

Opción 1: Registrar Alumno

```
PS C:\Users\adolf\Downloads\Actividad2\Actividad2> python cliente.py 192.168.0.252
--- MENÚ DE ALUMNOS ---
1. Registrar Alumno
2. Consultar Alumno
3. Actualizar Alumno
4. Eliminar Alumno
5. Buscar por criterio
6. Listar todos
0. Salir
Elige una opción: 1
Nombre: Rodrigo
Apellido: Lugo Manzano
Edad: 21
Curso: Sistemas Distribuidos
Dirección: Guillermo Prieto 73, Campamento 2 de Octubre, Iztacalco, CDMX (CP 08930)

  ========================================
  Alumno registrado correctamente.
   ID GENERADO: 7917ffac
  ========================================
  Este id fue generado automaticamente.

Alumno registrado correctamente. ID asignado: 7917ffac
```

Figura 18: Registrar Alumno.

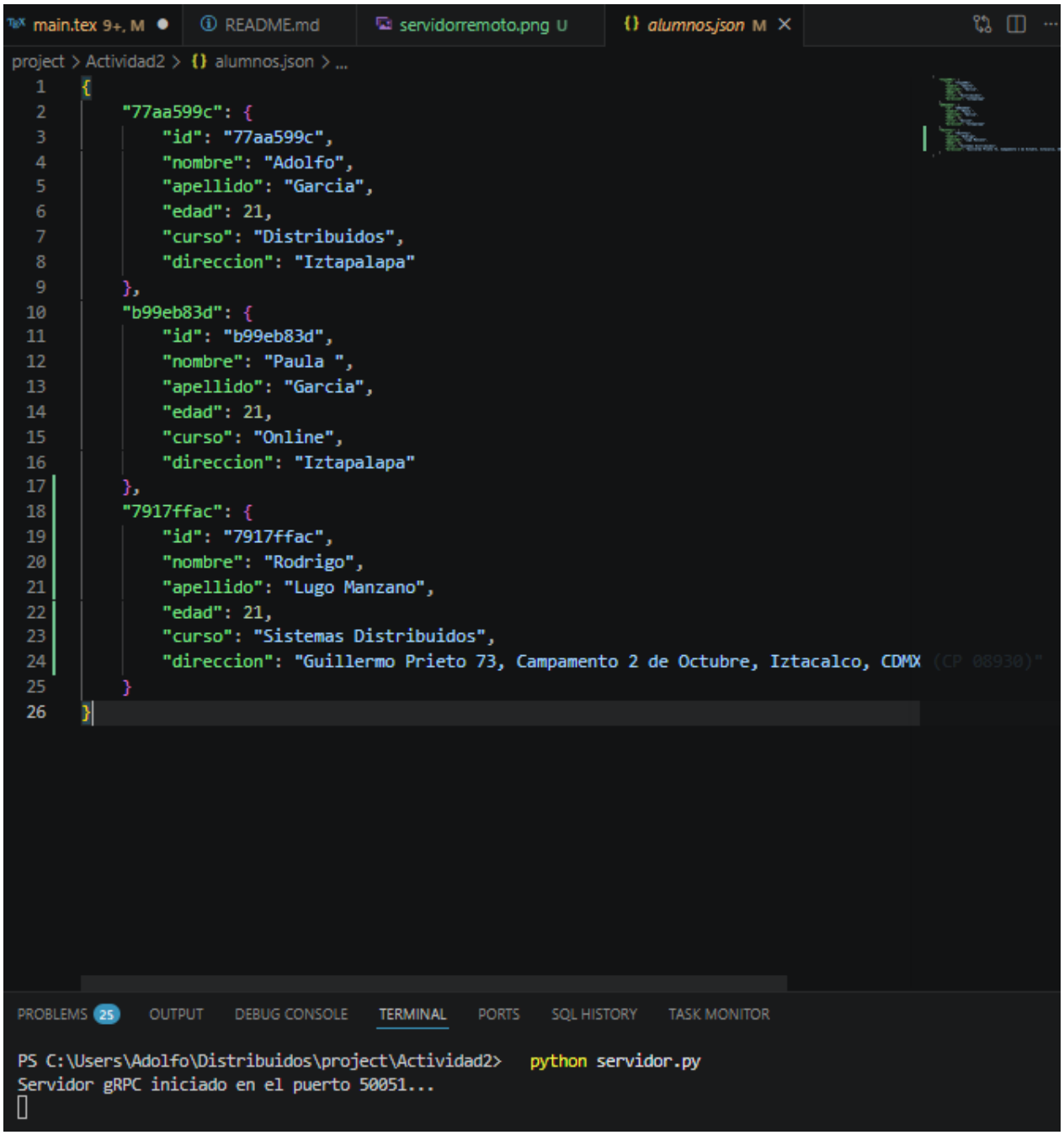

Figura 19: Cambios en el archivo JSON en el servidor de la Opción 1.

Opción 3: Eliminar Alumno

```
PS C:\Users\adolf\Downloads\Actividad2\Actividad2> python cliente.py 192.168.0.252
  Este id fue generado automaticamente.

Alumno registrado correctamente. ID asignado: 7917ffac

--- MENÚ DE ALUMNOS ---
1. Registrar Alumno
2. Consultar Alumno
3. Actualizar Alumno
4. Eliminar Alumno
5. Buscar por criterio
6. Listar todos
0. Salir
Elige una opción: 4
ID del alumno a eliminar: 7917ffac

Alumno eliminado.
```

Figura 20: Eliminar Alumno.

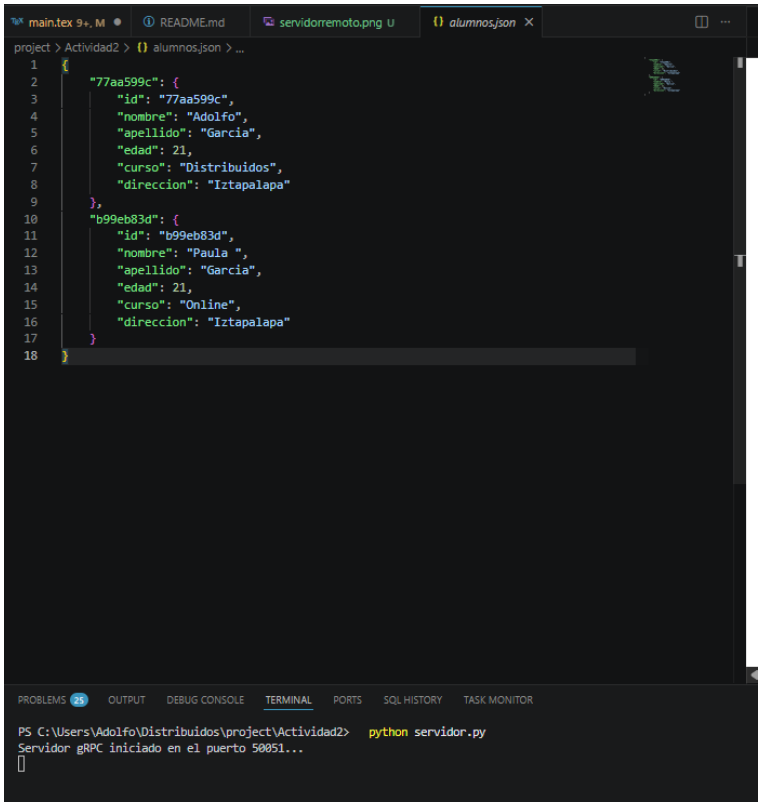

Figura 21: Cambios en el archivo JSON en el servidor de la Opción 3.

Referencias

Amazon Web Services. (s.f.). *Manage Amazon EC2 instance clock accuracy using Amazon Time Sync Service*. https://aws.amazon.com/blogs/mt/manage-amazon-ec2-instance-clock-accuracy-using-amazon-time-sync-service-and-amazon-cloudwatch-part-1/

Cisco Systems. (2026, 17 de febrero). *NTP Configuration Guide - Network Time Protocol [Cisco IOS XE 17]*. https://www.cisco.com/c/en/us/td/docs/switches/lan/c9000/time-sync/ntp-configuration-guide/m-network-time-protocol.html

Schmuck, F., & Cristian, F. (1990). *Probabilistic approach to distributed clock synchronization*. https://research.ibm.com/publications/probabilistic-approach-to-distributed-clock-synchronization